

LAPORAN PRAKTIKUM PEKAN 5

STRUKTUR DATA

“Double Linked List”

disusun Oleh:

NABIL FIKRI

2511533011

Dosen Pengampu:

Dr. WAHYUDI, S.T, M.T

Asisten Praktikum:

SHERLY SUKMADIRA



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa, karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum mata kuliah **Struktur Data** ini tepat pada waktunya.

Laporan ini disusun sebagai bentuk dokumentasi dan pertanggungjawaban atas pelaksanaan praktikum mengenai implementasi kelas dan objek melalui studi kasus pembuatan **Double Linked List** menggunakan bahasa pemrograman Java. Penulis menyadari bahwa keberhasilan penyusunan laporan ini tidak terlepas dari bantuan berbagai pihak. Oleh karena itu, penulis ingin menyampaikan terima kasih kepada:

1. Dr. Wahyudi S.T,M.T, selaku dosen mata kuliah yang telah memberikan bimbingan teori di kelas.
2. Kakak asisten praktikum yang telah memberikan arahan dan bantuan teknis selama proses praktikum berlangsung di laboratorium.
3. Rekan-rekan mahasiswa angkatan 2025 Informatika Universitas Andalas yang telah saling mendukung dalam proses belajar.

Penulis menyadari bahwa laporan ini masih jauh dari kesempurnaan, baik dari segi materi maupun sistematika penulisannya. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi perbaikan di masa mendatang. Akhir kata, semoga laporan ini dapat memberikan manfaat bagi pembaca dan perkembangan ilmu pengetahuan, khususnya di bidang Informatika.

Padang, 12 Mei 2026

Nabil Fikri

DAFTAR ISI

KATA PENGANTAR.....	i
BAB I PENDAHULUAN.....	1
1.1 Latar belakang.....	1
1.2 Tujuan Praktikum.....	1
BAB II PEMBAHASAN	2
2.1 <i>Mengenal Double Linked List</i>	2
2.1.1 <i>Pengertian Double Linked List</i>	2
2.1.2 Kelebihan dan Kekurangan.....	2
2.2 <i>Implementasi Double Linked List</i>	3
2.2.1 NodeDLL_2511533011.....	3
2.2.2 InsertDLL_2511533011	5
2.2.3 HapusDLL_2511533011	9
2.2.4 PenelusuranDLL_2511533011.....	12
BAB III PENUTUP	14
3.1 Kesimpulan	14
3.2 Saran.....	14
DAFTAR PUSTAKA.....	15
TUGAS PRAKTIKUM.....	16

BAB I

PENDAHULUAN

1.1 Latar belakang

Pemilihan struktur data yang tepat merupakan faktor kunci dalam efisiensi performa perangkat lunak. Salah satu bentuk struktur data linear yang fundamental adalah *Linked List*. Namun, pada *Single Linked List* standar, navigasi data hanya dapat dilakukan satu arah (maju), yang seringkali menjadi kendala ketika sistem membutuhkan akses data secara bolak-balik atau penghapusan node di posisi tertentu yang efisien.

Double Linked List hadir sebagai solusi atas keterbatasan tersebut. Dengan menambahkan satu pointer tambahan (*prev*) pada setiap *node*, struktur data ini memungkinkan navigasi dua arah, baik dari head ke tail maupun sebaliknya. Keunggulan utama dari *Double Linked List* terletak pada kemudahannya dalam memanipulasi data seperti penyisipan (*insertion*) dan penghapusan (*deletion*) di posisi manapun dalam list tanpa harus menelusuri ulang dari awal. Oleh karena itu, memahami mekanisme kerja pointer dalam *Double Linked List* menjadi sangat krusial bagi praktikan untuk membangun aplikasi yang memerlukan manajemen data dinamis yang fleksibel dan efisien.

1.2 Tujuan Praktikum

1. Mahasiswa dapat memahami konsep dasar *Double Linked List*.
2. Mahasiswa dapat menguasai operasi manipulasi data.
3. Mahasiswa dapat menerapkan navigasi dua arah.
4. Mahasiswa dapat menganalisis performa.

BAB II

PEMBAHASAN

2.1 Mengenal Double Linked List

2.1.1 Pengertian Double Linked List

Double Linked List adalah sekumpulan elemen (node) yang terhubung secara linear, di mana setiap node memiliki tiga bagian utama :

1. **Data** : Menyimpan nilai atau informasi.
2. **Pointer Next** : Menyimpan alamat node berikutnya.
3. **Pointer Prev** : Menyimpan alamat node sebelumnya.

Strukturnya:

NULL $\leftarrow$ Node1 $\leftrightarrow$ Node2 $\leftrightarrow$ Node3 $\rightarrow$ NULL

Dengan dua arah ini, navigasi menjadi lebih fleksibel.

2.1.2 Kelebihan dan Kekurangan

Keunggulan utamanya dibandingkan *Single Linked List* adalah kemampuan untuk melakukan penelusuran dua arah dan kemudahan dalam menghapus node tertentu karena kita memiliki akses langsung ke node sebelumnya.

1. Kelebihan
 - Bisa bergerak maju dan mundur
 - Penghapusan node lebih mudah
 - Navigasi lebih fleksibel

2. Kekurangan

- Membutuhkan memori lebih besar
- Struktur lebih kompleks

2.2 Implementasi Double Linked List

2.2.1 NodeDLL_2511533011

```

1 package pekan6_2511533011;
2
3 public class NodeDLL_2511533011 {
4     // mendefinisikan kelas node
5
6     int data_3011; // data
7     NodeDLL_2511533011 next_3011; // Pointer ke next node
8     NodeDLL_2511533011 prev_3011; // Pointer ke previous node
9
10    // Konstruktor
11    public NodeDLL_2511533011 (int data) {
12        this.data_3011 = data;
13        this.next_3011 = null;
14        this.prev_3011 = null;
15    }
16
17 }

```

Gambar 2.2.1

Class **NodeDLL_2511533011** ini adalah blok bangunan dasar (fondasi) dari sebuah **Double Linked List**. Di dalam struktur data, *Node* bertindak sebagai kontainer yang menyimpan nilai dan informasi tentang lokasinya dalam antrean.

Berikut adalah penjelasan detail mengenai komponen di dalam class tersebut :

1. Atribut (Variabel Anggota)

- **int data_3011** : Variabel ini berfungsi untuk menyimpan nilai utama yang ingin kamu masukkan ke dalam list. Dalam hal ini, tipe datanya adalah int (bilangan bulat).

- **NodeDLL_2511533011 next_3011** : Ini adalah pointer atau referensi yang menyimpan alamat memori dari **node setelahnya**. Ini yang membuat list bisa terhubung ke depan.
- **NodeDLL_2511533011 prev_3011** : Inilah ciri khas dari *Double Linked List*. Pointer ini menyimpan alamat memori dari **node sebelumnya**, sehingga memungkinkan kita untuk melakukan navigasi mundur (backward).

2. Konstruktor

- **public NodeDLL_2511533011 (int data)** : Saat kamu membuat objek node baru (misalnya saat melakukan *insert*), konstruktor ini akan dijalankan.
- **this.data_3011 = data**: Mengisi atribut data dengan nilai yang diterima dari parameter.
- **this.next_3011 = null & this.prev_3011 = null** : Secara default, saat node baru dibuat, ia berdiri sendiri dan belum terhubung ke node mana pun, sehingga kedua pointernya diatur ke null.

Penggunaan atribut **prev_3011** dan **next_3011** secara bersamaan memungkinkan struktur data ini memiliki fleksibilitas lebih tinggi dibandingkan *Single Linked List*. Dengan adanya referensi ke node sebelumnya, operasi penghapusan node di posisi tertentu menjadi lebih efisien karena program tidak perlu melakukan penelusuran ulang dari awal (**head**) untuk mencari node sebelum node yang akan dihapus.

2.2.2 InsertDLL_2511533011

```

1 package pekan6_2511533011;
2
3 public class InsertDLL_2511533011 {
4     // menambahkan node di awal dll
5     static NodeDLL_2511533011 insertBegin_3011 (NodeDLL_2511533011 head_3011, int data_3011) {
6         // buat node baru
7         NodeDLL_2511533011 new_node_3011 = new NodeDLL_2511533011 (data_3011);
8         // jadikan pointer nextnya head
9         new_node_3011.next_3011 = head_3011;
10        // jadikan pointer prev head ke new node
11        if (head_3011 != null) {
12            head_3011.prev_3011 = new_node_3011;
13        }
14        return new_node_3011;
15    }
16    // fungsi menambahkan node di akhir
17    public static NodeDLL_2511533011 insertEnd_3011 (NodeDLL_2511533011 head_3011, int newData_3011) {
18        // buat node baru
19        NodeDLL_2511533011 newNode_3011 = new NodeDLL_2511533011 (newData_3011);
20        // jika dll null jadikan head
21        if (head_3011 == null) {
22            head_3011 = newNode_3011;
23        } else {
24            NodeDLL_2511533011 curr_3011 = head_3011;
25            while (curr_3011.next_3011 != null) {
26                curr_3011 = curr_3011.next_3011;
27            }
28            curr_3011.next_3011 = newNode_3011;
29            newNode_3011.prev_3011 = curr_3011;
30        }
31        return head_3011;
32    }
33    //fungsi menambahkan node di posisi tertentu
34    public static NodeDLL_2511533011 insertAtPosition (NodeDLL_2511533011 head_3011, int pos_3011, int new_data_3011) {
35        // buat node baru
36        NodeDLL_2511533011 new_node_3011 = new NodeDLL_2511533011 (new_data_3011);
37        if (pos_3011 == 1) {
38            new_node_3011.next_3011 = head_3011;
39            if (head_3011 != null) {
40                head_3011.prev_3011 = new_node_3011;
41            }
42            head_3011 = new_node_3011;

```

```

43         return head_3011;
44     }
45     NodeDLL_2511533011 curr_3011 = head_3011;
46     for (int i=1 ; i < pos_3011 - 1 && curr_3011 != null; i++) {
47         curr_3011 = curr_3011.next_3011;
48     }
49     if (curr_3011 == null) {
50         System.out.println("Posisi tidak ada");
51         return head_3011;
52     }
53     new_node_3011.prev_3011 = curr_3011;
54     new_node_3011.next_3011 = curr_3011.next_3011;
55     curr_3011.next_3011 = new_node_3011;
56     if (new_node_3011.next_3011 != null) {
57         new_node_3011.next_3011.prev_3011 = new_node_3011;
58     }
59     return head_3011;
60 }
61 public static void printList_3011 (NodeDLL_2511533011 head_3011) {
62     NodeDLL_2511533011 curr_3011 = head_3011;
63     while (curr_3011 != null) {
64         System.out.print(curr_3011.data_3011 + " <-> ");
65         curr_3011 = curr_3011.next_3011;
66     }
67     System.out.println();
68 }
69 public static void main (String [] args) {
70     // membuat dll 2 <-> 3 <-> 5
71     NodeDLL_2511533011 head_3011 = new NodeDLL_2511533011 (2);
72     head_3011.next_3011 = new NodeDLL_2511533011 (3);
73     head_3011.next_3011.prev_3011 = head_3011;
74     head_3011.next_3011.next_3011 = new NodeDLL_2511533011 (5);
75     head_3011.next_3011.next_3011.prev_3011 = head_3011.next_3011;
76     // cetak dll awal
77     System.out.print("DLL Awal: ");
78     printList_3011(head_3011);
79     // tambah 1 di awal
80     head_3011 = insertBegin_3011 (head_3011,1);
81     System.out.println("Simpul 1 ditambah di awal : ");
82     printList_3011 (head_3011);
83     // tambah 6 di akhir
84     System.out.println("simpul 6 di tambah di akhir: ");
85     int data_3011 = 6;
86     head_3011 = insertEnd_3011 (head_3011, data_3011);
87     printList_3011 (head_3011);
88     // menambahkan node 4 di posisi 4
89     System.out.println("tambah node 4 di posisi 4: ");
90     int data2_3011 = 4;
91     int pos_3011 = 4;
92     head_3011 = insertAtPosition (head_3011, pos_3011, data2_3011);
93     printList_3011(head_3011);
94 }
95
96 }

```

Gambar 2.2.2

Class **InsertDLL_2511533011** adalah class kontroler yang bertanggung jawab untuk mengelola operasi penambahan (*insertion*) data ke dalam list. Class ini menggunakan metode static agar fungsinya bisa dipanggil tanpa perlu menginstansiasi objek class tersebut.

Berikut detail logika yang digunakan dalam class ini :

1. Metode **insertBegin_3011** (Tambah di Depan)

Metode ini bertujuan menempatkan data baru sebagai elemen pertama (head).

- **Logika:** Node baru dibuat, lalu next dari node baru diarahkan ke head yang lama. Jika list tidak kosong, prev dari head lama harus menunjuk ke node baru.
- **Output:** Mengembalikan node baru tersebut sebagai head yang baru.

2. Metode **insertEnd_3011** (Tambah di Belakang)

Digunakan untuk menambahkan data di urutan paling terakhir.

- **Logika:** Jika list kosong, node baru langsung jadi head. Jika tidak, program melakukan perulangan (*traversal*) menggunakan while hingga menemukan node terakhir (di mana `next == null`). Setelah ketemu, pointer next dari node terakhir tersebut dihubungkan ke node baru, dan prev dari node baru dihubungkan kembali ke node terakhir tadi.

3. Metode **insertAtPosition** (Tambah di Posisi Tertentu)

Ini adalah bagian yang paling kompleks karena melibatkan penyisipan di tengah.

- **Logika:**
 - Jika posisi adalah 1, ia memanggil logika tambah di depan.
 - Jika posisi di tengah, program melakukan *loop* untuk mencari node tepat **sebelum** posisi yang diinginkan.

- Pentingnya *Double Linked List* terlihat di sini: saat node baru disisipkan, ada 4 koneksi pointer yang diperbarui agar rantai tidak putus (2 pointer next dan 2 pointer prev).

4. Metode `printList_3011` & `main`

- **`printList_3011`**: Fungsi bantuan untuk mencetak isi list ke layar dengan format data <-> data agar visualisasi *double link*- nya terlihat jelas.
- **`main`**: Sebagai tempat pengujian (test case). Kamu membuat list awal secara manual (2 <-> 3 <-> 5), lalu mendemonstrasikan ketiga fungsi penyisipan di atas secara berurutan.

Dalam implementasi **`insertAtPosition`**, kode ini telah menangani kondisi kritis di mana posisi yang diminta lebih besar dari jumlah node yang ada (**`if (curr_3011 == null)`**). Selain itu, penanganan **`if (new_node_3011.next_3011 != null)`** memastikan bahwa program tidak akan mengalami *NullPointerException* saat mencoba mengakses pointer **`prev`** dari node yang tidak ada (jika node baru adalah node terakhir).

2.2.3 HapusDLL_2511533011

```

1 package pekan6_2511533011;
2
3 public class HapusDLL_2511533011 {
4     // fungsi menghapus node awal
5     public static NodeDLL_2511533011 delHead_2511533011(NodeDLL_2511533011 head_3011) {
6         if (head_3011 == null) {
7             return null;
8         }
9         NodeDLL_2511533011 temp_3011 = head_3011;
10        head_3011 = head_3011.next_3011;
11        if (head_3011 != null) {
12            head_3011.prev_3011 = null;
13        }
14        return head_3011;
15    }
16    // fungsi menghapus di akhir
17    public static NodeDLL_2511533011 dellast_2511533011(NodeDLL_2511533011 head_3011) {
18        if (head_3011 == null) {
19            return null;
20        }
21        if (head_3011.next_3011 == null) {
22            return null;
23        }
24        NodeDLL_2511533011 curr_3011 = head_3011;
25        while (curr_3011.next_3011 != null) {
26            curr_3011 = curr_3011.next_3011;
27        }
28        // update pointer previous node
29        if (curr_3011.prev_3011 != null) {
30            curr_3011.prev_3011.next_3011 = null;
31        }
32        return head_3011;
33    }
34    // fungsi menghapus node di posisi tertentu
35    public static NodeDLL_2511533011 delPos_2511533011(NodeDLL_2511533011 head_3011, int pos_3011) {
36        // jika DLL kosong
37        if (head_3011 == null) {
38            return head_3011;
39        }
40        NodeDLL_2511533011 curr_3011 = head_3011;
41        // telusuri sampai node yang akan dihapus
42        // telusuri sampai node yang akan dihapus
43        for (int i = 1; curr_3011 != null && i < pos_3011; ++i) {
44            curr_3011 = curr_3011.next_3011;
45        }
46        // jika posisi tidak ditemukan
47        if (curr_3011 == null) {
48            return head_3011;
49        }
50        // update pointer
51        if (curr_3011.prev_3011 != null) {
52            curr_3011.prev_3011.next_3011 = curr_3011.next_3011;
53        }
54        if (curr_3011.next_3011 != null) {
55            curr_3011.next_3011.prev_3011 = curr_3011.prev_3011;
56        }
57        // jika yang dihapus head
58        if (head_3011 == curr_3011) {
59            head_3011 = curr_3011.next_3011;
60        }
61        return head_3011;
62    }
63    // fungsi mencetak DLL
64    public static void printList_2511533011 (NodeDLL_2511533011 head_3011) {
65        NodeDLL_2511533011 curr_3011 = head_3011;
66        while (curr_3011 != null) {
67            System.out.print(curr_3011.data_3011 + " ");
68            curr_3011 = curr_3011.next_3011;
69        }
70        System.out.println();
71    }
72    public static void main(String[] args) {
73        // buat sebuah DLL
74        NodeDLL_2511533011 head_3011 = new NodeDLL_2511533011(1);
75        head_3011.next_3011 = new NodeDLL_2511533011(2);
76        head_3011.next_3011.prev_3011 = head_3011;
77        head_3011.next_3011.next_3011 = new NodeDLL_2511533011(3);
78        head_3011.next_3011.next_3011.prev_3011 = head_3011.next_3011;
79        head_3011.next_3011.next_3011.next_3011 = new NodeDLL_2511533011(4);
80        head_3011.next_3011.next_3011.next_3011.prev_3011 = head_3011.next_3011.next_3011;
81        head_3011.next_3011.next_3011.next_3011.next_3011 = new NodeDLL_2511533011(5);
82        head_3011.next_3011.next_3011.next_3011.next_3011.prev_3011 = head_3011.next_3011.next_3011.next_3011;
83    }

```

```

84     System.out.print("DLL Awal : ");
85     printList_2511533011(head_3011);
86
87     System.out.print("Setelah head dihapus : ");
88     head_3011 = delHead_2511533011(head_3011);
89     printList_2511533011(head_3011);
90
91     System.out.print("Setelah node terakhir dihapus : ");
92     head_3011 = delLast_2511533011(head_3011);
93     printList_2511533011(head_3011);
94
95     System.out.print("menghapus node ke 2 : ");
96     head_3011 = delPos_2511533011(head_3011, 2);
97     printList_2511533011(head_3011);
98 }
99 }

```

Gambar 2.2.3

Class **HapusDLL_2511533011** adalah komponen krusial dalam manajemen memori Double Linked List. Tugas utamanya adalah memutuskan hubungan (*link*) pada node tertentu agar node tersebut terlepas dari rantai list dan akhirnya dihapus oleh *Garbage Collector* Java.

Berikut penjelasannya :

1. Logika Utama Penghapusan

Berbeda dengan *Single Linked List*, pada *Double Linked List*, kita harus memperbarui **dua arah pointer** (next dan prev) agar integritas list tetap terjaga setelah ada node yang hilang.

- **delHead_2511533011:**
 - Menggeser head ke node berikutnya (head.next).
 - Penting: Pointer prev pada head yang baru harus diatur ke null karena sekarang ia adalah elemen pertama.
- **delLast_2511533011:**
 - Melakukan *traversal* sampai menemukan node terakhir.

- Menghapus referensi next dari node tepat sebelum node terakhir (dibuat menjadi null).
- **delPos_2511533011:**
 - Ini adalah metode yang paling fleksibel. Program mencari node di posisi i melalui perulangan.
 - **Pembaruan Pointer:** Inilah inti dari *Double Linked List*.
 - `curr.prev.next = curr.next` (Menghubungkan node sebelumnya ke node setelah node yang dihapus).
 - `curr.next.prev = curr.prev` (Menghubungkan node setelahnya kembali ke node sebelumnya)

Operasi penghapusan pada *Double Linked List* menunjukkan efisiensi dalam hal manipulasi pointer. Jika pada *Single Linked List* kita membutuhkan akses ke node sebelumnya (yang biasanya dilakukan dengan menyimpan referensi **prev** secara manual saat *traversal*), pada *Double Linked List* kita sudah memiliki pointer **prev** yang tersedia langsung di dalam node. Hal ini menyederhanakan logika penghapusan, terutama pada fungsi **delPos_2511533011**, karena kita dapat langsung mengakses `curr.prev` tanpa harus mengulang pencarian dari **head**.

2.2.4 PenelusuranDLL_2511533011

```

1 package pekan6_2511533011;
2
3 public class PenelusuranDLL_2511533011 {
4     // fungsi penelusuran maju
5     static void forwardTraversal_2511533011(NodeDLL_2511533011 head_3011) {
6         // mulai penelusuran dari head
7         NodeDLL_2511533011 curr_3011 = head_3011;
8         // lanjutkan sampai akhir
9         while (curr_3011 != null) {
10             // print data
11             System.out.print(curr_3011.data_3011 + " <-> ");
12             // pindah ke node berikutnya
13             curr_3011 = curr_3011.next_3011;
14         }
15         // print spasi
16         System.out.println();
17     }
18     // fungsi penelusuran mundur
19     static void backwardTraversal_2511533011(NodeDLL_2511533011 tail_3011) {
20         // mulai dari akhir
21         NodeDLL_2511533011 curr_3011 = tail_3011;
22         // lanjut sampai head
23         while (curr_3011 != null) {
24             // cetak data
25             System.out.print(curr_3011.data_3011 + " <-> ");
26             // pindah ke node sebelumnya
27             curr_3011 = curr_3011.prev_3011;
28         }
29         // cetak spasi
30         System.out.println();
31     }
32
33     public static void main(String[] args) {
34         // cetak DLL
35         NodeDLL_2511533011 head_3011 = new NodeDLL_2511533011(1);
36         NodeDLL_2511533011 second_3011 = new NodeDLL_2511533011(2);
37         NodeDLL_2511533011 third_3011 = new NodeDLL_2511533011(3);
38
39         head_3011.next_3011 = second_3011;
40         second_3011.prev_3011 = head_3011;
41         second_3011.next_3011 = third_3011;
42         third_3011.prev_3011 = second_3011;
43
44         System.out.println("Penelusuran maju : ");
45         forwardTraversal_2511533011(head_3011);
46
47         System.out.println("Penelusuran mundur : ");
48         backwardTraversal_2511533011(third_3011);
49     }
50 }

```

Gambar 2.2.4

Class **PenelusuranDLL_2511533011** adalah implementasi dari fitur navigasi dua arah yang menjadi keunggulan utama dari *Double Linked List*. Di sini, kita tidak hanya bisa bergerak dari depan ke belakang, tetapi juga sebaliknya.

Berikut penjelasan dari isi class tersebut :

1. Navigasi Dua Arah

- **forwardTraversal_2511533011 (Maju):**
 - **Logika:** Dimulai dari head, program menelusuri list dengan mengakses pointer next_3011 secara berulang sampai menemukan null (akhir dari list). Ini adalah cara standar yang juga dimiliki oleh *Single Linked List*.
- **backwardTraversal_2511533011 (Mundur):**
 - **Logika:** Inilah keunggulan dari *Double Linked List*. Program dimulai dari node tail (akhir), lalu menelusuri list dengan mengakses pointer prev_3011 untuk mundur ke node sebelumnya sampai mencapai head (atau saat pointer prev bernilai null).

Kemampuan untuk melakukan penelusuran mundur (**backwardTraversal**) memberikan fleksibilitas akses data yang tidak dimiliki oleh *Single Linked List*. Dalam sistem nyata, fitur ini sering digunakan pada aplikasi yang memerlukan riwayat navigasi, seperti tombol 'Back' pada peramban web (browser) atau fungsi 'Undo' pada perangkat lunak pengolah kata, di mana sistem perlu kembali ke status sebelumnya dengan efisien.

BAB III

PENUTUP

3.1 Kesimpulan

Praktikum ini berhasil mengimplementasikan struktur data *Double Linked List* yang memiliki keunggulan utama berupa fleksibilitas navigasi dua arah dan efisiensi dalam manipulasi data. Melalui penggunaan dua pointer, yakni *next* untuk menunjuk ke node berikutnya dan *prev* untuk menunjuk ke node sebelumnya, setiap operasi penyisipan dan penghapusan menjadi lebih sederhana karena akses ke node sebelumnya tidak memerlukan penelusuran ulang dari awal list. Meskipun struktur ini memberikan performa yang lebih baik dalam skenario tertentu, konsekuensinya adalah penggunaan memori yang lebih besar dibandingkan *Single Linked List* dikarenakan setiap node harus menyimpan dua referensi alamat pointer. Secara keseluruhan, pemahaman terhadap manajemen pointer dalam *Double Linked List* merupakan dasar penting bagi pengembangan struktur data yang lebih kompleks seperti *Doubly Linked Queue* atau *Stack* dua arah.

3.2 Saran

Berdasarkan hasil praktikum yang telah dilakukan, terdapat beberapa saran untuk pengembangan dan pemahaman lebih lanjut. Pertama, dalam penulisan kode di masa depan, disarankan untuk mengimplementasikan *dummy node* atau *sentinel node* pada bagian head dan tail untuk menyederhanakan pengecekan kondisi *null* saat melakukan operasi penyisipan atau penghapusan, sehingga kode menjadi lebih bersih dan minim kesalahan (*error-prone*). Kedua, praktikan disarankan untuk lebih memperdalam pemahaman mengenai alokasi memori dinamis di Java, terutama bagaimana *Garbage Collector* bekerja setelah sebuah node diputus koneksinya dari rantai *Double Linked List*.

DAFTAR PUSTAKA

- [1] dianaape, "Linked List: Single, Double, dan Circular Linked List," Telkom University Jakarta, [Online]. Available: <https://jakarta.telkomuniversity.ac.id/linked-list-single-double-dan-circular-linked-list/>. [Accessed 13 Mei 2026].

TUGAS PRAKTIKUM

PEKAN 6

Buatlah program sederhana untuk mengelola playlist lagu menggunakan Double Linked List. Setiap lagu adalah node yang memiliki pointer ke lagu sebelumnya (prev) dan lagu berikutnya (next). Program harus memungkinkan pengguna menambah lagu, menghapus lagu, dan melihat daftar lagu dari dua arah.

1. Class ADT **Lagu_2511533011**

```
1 package pekan6_2511533011;
2
3 public class Lagu_2511533011 {
4     String judul_3011;
5     String penyanyi_3011;
6     Lagu_2511533011 next_3011;
7     Lagu_2511533011 prev_3011;
8
9     // Constructor
10    public Lagu_2511533011(String judul_3011, String penyanyi_3011) {
11        this.judul_3011 = judul_3011;
12        this.penyanyi_3011 = penyanyi_3011;
13        this.next_3011 = null;
14        this.prev_3011 = null;
15    }
16
17    // Getter
18    public String getJudul_3011() {
19        return judul_3011;
20    }
21    public String getPenyanyi_3011() {
22        return penyanyi_3011;
23    }
24
25    // Setter
26    public void setJudul_3011 (String judul_3011) {
27        this.judul_3011 = judul_3011;
28    }
29    public void setPenyanyi_3011 (String penyanyi_3011) {
30        this.penyanyi_3011 = penyanyi_3011;
31    }
32 }
```

2. Class main Musik_2511533011

```
1 package pekan6_2511533011;
2 import java.util.Scanner;
3
4 public class Musik_2511533011 {
5     private Lagu_2511533011 head_3011;
6     private Lagu_2511533011 tail_3011;
7
8     // 1. Tambah Lagu di Akhir
9     public void tambahLagu_3011(String judul_3011, String penyanyi_3011) {
10         Lagu_2511533011 baru = new Lagu_2511533011(judul_3011, penyanyi_3011);
11         if (head_3011 == null) {
12             head_3011 = tail_3011 = baru;
13         } else {
14             tail_3011.next_3011 = baru;
15             baru.prev_3011 = tail_3011;
16             tail_3011 = baru;
17         }
18         System.out.println("Lagu berhasil ditambahkan!");
19     }
20
21     // 2. Hapus Lagu Awal
22     public void hapusLaguAwal_3011() {
23         if (head_3011 == null) {
24             System.out.println("Playlist kosong!");
25         } else {
26             head_3011 = head_3011.next_3011;
27             if (head_3011 != null) head_3011.prev_3011 = null;
28             else tail_3011 = null;
29             System.out.println("Lagu pertama berhasil dihapus.");
30         }
31     }
32
33     // 3. Tampil Maju
34     public void tampilMaju_3011() {
35         if (head_3011 == null) System.out.println("Playlist kosong.");
36         Lagu_2511533011 curr = head_3011;
37         while (curr != null) {
38             System.out.println(curr.judul_3011 + " - " + curr.penyanyi_3011);
39             curr = curr.next_3011;
40         }
41     }
42
43     // 4. Tampil Mundur
44     public void tampilMundur_3011() {
45         if (tail_3011 == null) System.out.println("Playlist kosong.");
46         Lagu_2511533011 curr = tail_3011;
47         while (curr != null) {
48             System.out.println(curr.judul_3011 + " - " + curr.penyanyi_3011);
49             curr = curr.prev_3011;
50         }
51     }
52
53     // 5. Cari Lagu
54     public void cariLagu_3011(String judul_3011) {
55         Lagu_2511533011 curr = head_3011;
56         boolean found = false;
57         while (curr != null) {
58             if (curr.judul_3011.equalsIgnoreCase(judul_3011)) {
59                 System.out.println("Lagu ditemukan: " + curr.judul_3011 + " oleh " + curr.penyanyi_3011);
60                 found = true;
61             }
62             curr = curr.next_3011;
63         }
64         if (!found) System.out.println("Lagu tidak ditemukan.");
65     }
66
67     public static void main(String[] args) {
68         Musik_2511533011 playlist = new Musik_2511533011();
69         Scanner sc = new Scanner(System.in);
70         int pilihan = 0;
```

```

71
72 while (pilihan != 6) {
73     System.out.println("\n=== Playlist Musik NIM: 2511533011 ===");
74     System.out.println("1. Tambah Lagu");
75     System.out.println("2. Hapus Lagu Pertama");
76     System.out.println("3. Lihat Playlist (Maju)");
77     System.out.println("4. Lihat Playlist (Mundur)");
78     System.out.println("5. Cari Lagu");
79     System.out.println("6. Keluar");
80     System.out.print("Pilihan: ");
81     pilihan = sc.nextInt();
82     sc.nextLine();
83
84     switch (pilihan) {
85     case 1:
86         System.out.print("Judul: ");
87         String j = sc.nextLine();
88         System.out.print("Penyanyi: ");
89         String p = sc.nextLine();
90         playlist.tambahLagu_3011(j, p);
91         break;
92     case 2:
93         playlist.hapusLaguAwal_3011();
94         break;
95     case 3:
96         playlist.tampilMaju_3011();
97         break;
98     case 4:
99         playlist.tampilMundur_3011();
100        break;
101     case 5:
102        System.out.print("Masukkan judul lagu: ");
103        String cari = sc.nextLine();
104        playlist.cariLagu_3011(cari);
105        break;
106     case 6:
107        System.out.println("Program selesai.");
108        break;
109     default:
110        System.out.println("Pilihan tidak valid!");
111    }
112 }
113 sc.close();
114 }

```

- Kelas Lagu_2511533011: Berfungsi sebagai model data atau representasi dari satu unit informasi (node). Kelas ini berperan menyimpan atribut lagu dan menyediakan mekanisme akses aman melalui *constructor*, *getter*, dan *setter*, yang memastikan data terenkapsulasi dengan baik sesuai standar pemrograman berorientasi objek.
- Kelas Musik_2511533011: Berfungsi sebagai kelas kontroler yang mengelola logika operasional dari *Double Linked List*. Kelas ini mengimplementasikan fungsi-fungsi utama seperti penyisipan, penghapusan, dan navigasi data, serta menyediakan antarmuka interaktif bagi pengguna untuk mengelola playlist lagu.